Introdução ao Spring e Spring MVC Forms

Conteudista: Prof. Me. Alexander Gobbato Paulino Albuquerque
Revisão Textual: Prof.ª Dra. Selma Aparecida Cesarin

Objetivo da Unidade:

- Conhecer o processo de criação de uma aplicação completa dentro dos padrões do *Spring*, utilizando o *framework Spring* MVC *Forms*.

Material Teórico

Material Complementar

Referências

Material Teórico

Introdução

Estamos em um ponto dos nossos estudos no qual já temos uma bagagem interessante de algumas técnicas e formas de programar.

Nesta Unidade, aumentaremos mais ainda o nosso conhecimento, pois agora iremos introduzir um *framework* utilizado no mundo coorporativo.

Introdução ao *Spring*

Entendendo o *Framework Spring*

Muitos já ouviram falar no *Spring Framework*, mas a complexidade inicial afasta ou assusta bastante os principiantes do mundo Java.

O *Spring Framework* pode parecer nada simples, mas iremos ver que ele não é um "bicho de sete cabeças".

Spring é um *framework* de código aberto (*open source*), criado por Rod Johnson, em meados de 2002, e apresentado no seu livro ***Expert One-on-One: JEE Design and Development***.

Foi criado com o intuito de simplificar a programação em *Java*, possibilitando construir aplicações que antes só eram possíveis utilizando EJBs.

Atualmente, o *Spring* tem diversos módulos, como ***Spring Data*** (trata da persistência) e ***Spring Security*** (trata da segurança da aplicação), entre outros

Entretanto, o principal (core) pode ser utilizado em qualquer aplicação *Java*, e suas principais funcionalidades são a injeção de dependência (CDI) e a programação orientada a aspectos (AOP).

Cabe ao desenvolvedor dizer ao *Spring* o que quer usar, o que faz dele uma poderosa ferramenta, pois não existe a necessidade de arrastar todas as ferramentas do *framework* para criar uma aplicação simples.

Atualmente, o *Spring Framework* é o *framework* mais usado no Mercado e você conhecerá todo o poder dele, iniciando com o acesso ao Banco de Dados, usando o *Spring Data*.

Uma vez dominado o Banco de Dados, você verá como construir uma aplicação *web* do zero, com o *Spring MVC*.

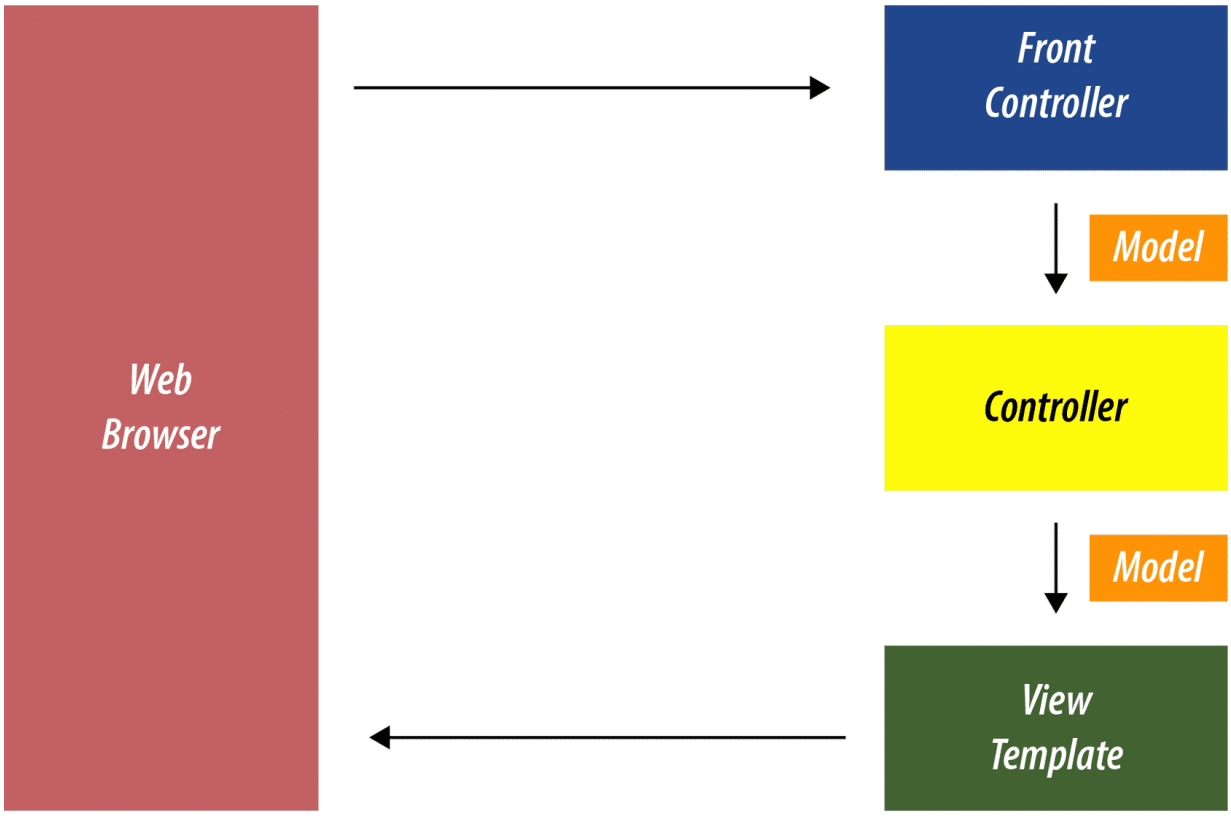


Figura 1 – Estrutura do *Spring MVC*

O que é o *Spring* MVC?

Como o nome sugere, é baseado no padrão de projeto *Model-View-Controller*.

Ele utiliza os recursos do *Core Spring Framework*, para que você possa aproveitar toda a sua inversão de controle e dependência de injeção, ou seja, você tem um pedido recebido vindo do navegador, ele encontrará o *Front Controller* do *Spring* MVC.

Esse processo delegará a solicitação para um código do *Controller*.

O código do *Controller* é um código criado por você, que contém sua lógica de negócios.

Então, você cria um modelo e o envia de volta para o *Front Controller* e, em seguida, o *Front Controller* passa esse modelo para o *View Template*.

Portanto, seu *View Template* é basicamente uma página *html* ou uma página *JSP* que pegará esses dados e renderizará uma resposta para o navegador.

Quais são os Benefícios de Usar o *Spring* MVC?

Basicamente, a maneira do *Spring* de construir aplicativos *web* usando *Java*.

Então, você pode aproveitar um conjunto de componentes de interface do usuário reutilizáveis e eles estão disponíveis na forma de *tags* personalizadas do *Spring* JSP.

Você também pode gerenciar seu estado de aplicativo para solicitações da *Web* para rastreamento de sessão ou rastreamento de aplicativo e também pode processar seus dados de formulário, para que também possa executar a validação nos dados de formulário, conversão, e assim por diante.

Por fim, o *Spring* MVC tem uma configuração muito flexível para as camadas de visualização. Portanto, você não está limitado a usar apenas JSP e pode usar outras camadas de visualização, como *Thymeleaf*, *Velocity* ou *FreeMarker*.

Quais são os Componentes de um Aplicativo *Spring* MVC?

É uma coleção de páginas da *web* para *layout* de seus componentes de interface do usuário. Ele tem também uma coleção de *beans Spring* para controlar e manipular serviços, e assim por diante, e, finalmente, tem a configuração de *Spring*.

Você pode escolher XML, Anotações ou *Java* puro e esses são os principais componentes de um aplicativo *Spring MVC*.

Entendendo o funcionamento do *Spring* MVC

Tudo começa com o primeiro pedido recebido e encontra algo chamado *Front Controller*.

O *Front Controller* é conhecido como o *DispactcherServlet* e faz parte do *framework Spring*.

Ele já foi desenvolvido pela Equipe de Desenvolvimento do *Spring* para que você não precise criá-lo. Isso faz parte dos arquivos .jar do *Spring*.

Então, fora da caixa, você tem o seu *Front Controller* e o que esse *Front Controller* fará é delegar o pedido para alguns outros objetos ou itens aqui em nosso sistema.

Como desenvolvedor, você criará o *Model object*, a *View Templates* e o *Controller* classes MVC.

Os *Model object* estão em laranja e contêm dados.

Os *View templates*, que estão em verde escuro, são as páginas *JSPs* ou sua página de visualização, para renderizar os dados.

As classes do *Controller*, que estão em amarelo, são a sua lógica de negócios real ou a sua lógica de processamento

Controller

Quando o *Front Controller* tem uma solicitação, ele delega a solicitação ao *Controller*.

O *Controller* é o código que você irá criar. Nesse *Controller*, teremos o negócio.

É aqui que você lidará com o pedido e no qual talvez você leia alguns dados de formulário e, então, você pegará esses dados e os armazenará ou recuperará.

Você pode armazená-los em um Banco de Dados ou recuperar informações de um serviço da *web*, basicamente. Depois de ter seus dados, você poderá pegá-los e colocá-los no *Model*.

O *Model* é apenas um contêiner para seus dados e, em seguida, você o repassa para o *Model* apropriado.

Assim, novamente, seu *Controller*, que contém sua lógica de negócios, lidará com a solicitação da *web*.

Model

Quando o ***Controller*** é desligado e executa uma operação para recuperar dados de um sistema *back end*, como um Banco de Dados ou serviço da *web*, ou um *Spring Bean*, você pode pegar esses dados e colocá-los no modelo.

Dessa forma, o ***model*** novamente é o seu contêiner, como sua mala ou sua bagagem, para o envio de dados entre várias partes do seu aplicativo *Spring MVC*.

Assim, os dados do modelo serão realmente passados para o ***View Template*** e eles podem realmente manipular isso para exibir os dados.

View Template

O *View Template* é o modelo de visualização mais comum que você usará (*JSP* e *JSTL*).

Esses dados de modelo chegam ao *View Template* e, em seguida, sua página *JSP* pode ler os dados do modelo e exibi-los.

Por exemplo, se você tem uma lista de alunos ou uma lista de produtos, o *JSP* pode criar uma tabela para exibir essa lista de produtos ou a lista de alunos.

Configurando o Ambiente

Neste ponto, precisamos aplicar algumas mudanças nos arquivos de configuração do projeto e também realizar alguns *downloads* para que o projeto tenha todas as referências necessárias para "rodar".

Site

repo.spring.io

Para baixar os arquivos necessário do *framework*, você deverá acessar o *link* a seguir e selecionar o arquivo spring-5.3.0-dist.zip.

Clique no botão para conferir o conteúdo.

ACESSE

Depois de descompactar o arquivo, copie o .jar para a pasta lib do seu projeto eclipse.

São vários arquivos e você pode copiar todos, mas, caso queira selecionar, veja a lista a seguir:

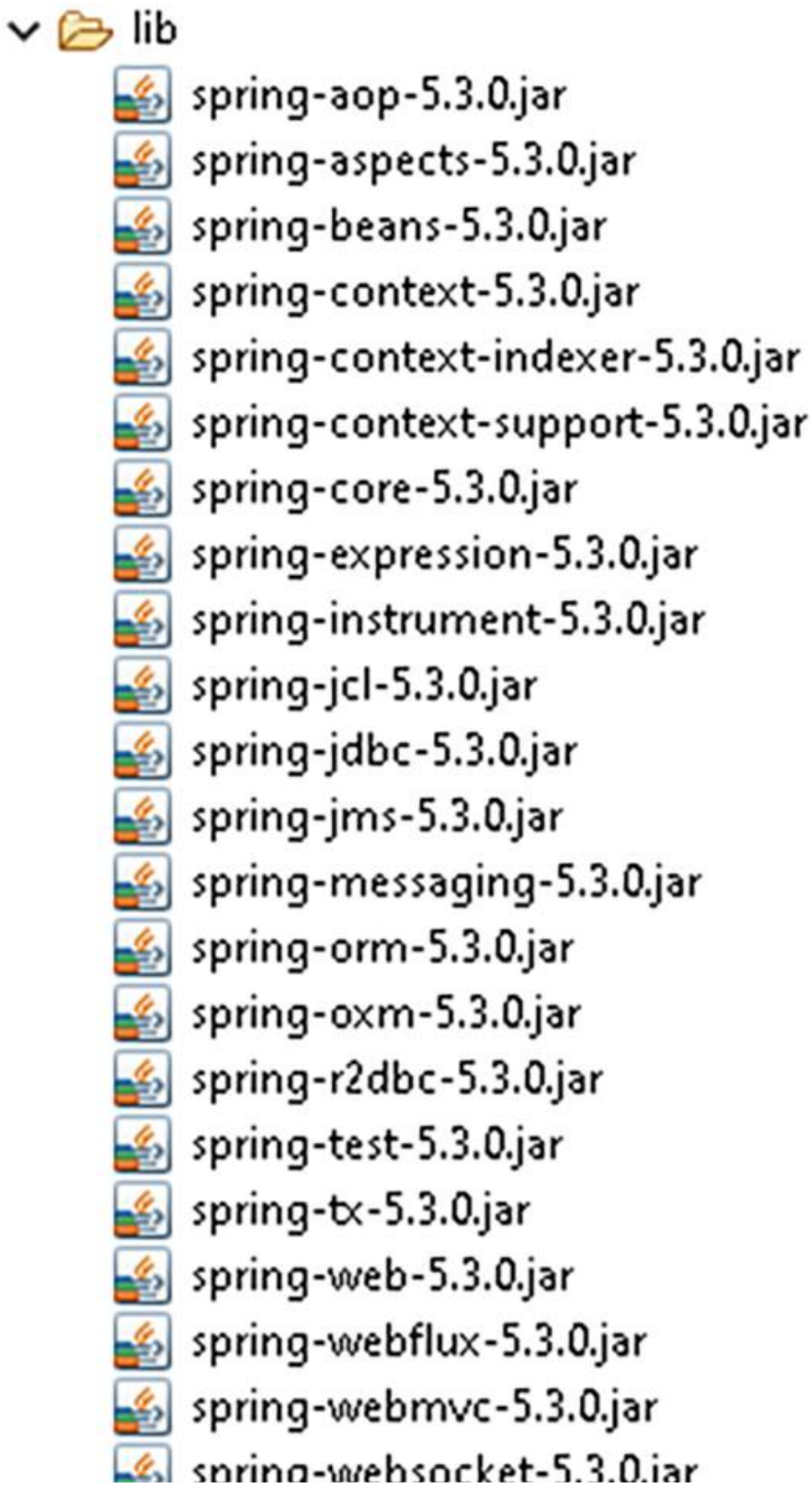

lib
spring-aop-5.3.0.jar
spring-aspects-5.3.0.jar
spring-beans-5.3.0.jar
spring-context-5.3.0.jar
spring-context-indexer-5.3.0.jar
spring-context-support-5.3.0.jar
spring-core-5.3.0.jar
spring-expression-5.3.0.jar
spring-instrument-5.3.0.jar
spring-jcl-5.3.0.jar
spring-jdbc-5.3.0.jar
spring-jms-5.3.0.jar
spring-messaging-5.3.0.jar
spring-orm-5.3.0.jar
spring-oxm-5.3.0.jar
spring-r2dbc-5.3.0.jar
spring-test-5.3.0.jar
spring-tx-5.3.0.jar
spring-web-5.3.0.jar
spring-webflux-5.3.0.jar
spring-webmvc-5.3.0.jar
spring-websocket-5.3.0.jar



Figura 2 – Estrutura dos arquivos para utilização do *framework*
Fonte: Reprodução

Para que o *Spring MVC* funcione, teremos de alterar o arquivo que se encontra na pasta *WEB-INF/web.xml*:

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
    id="WebApp_ID" version="3.1">
  <display-name>TesteSpring</display-name>
  <servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <init-param>
      <param-name>contextConfigLocation</param-name>
      <param-value>/WEB-INF/spring-mvc-servlet.xml</param-value>
    </init-param>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>/</url-pattern>
  </servlet-mapping>
</web-app>
```

Figura 3 – Configuração do *web.xml*
Fonte: Reprodução

Após a modificação do arquivo *web.xml* configurando o *framework*, precisamos criar um arquivo, que chamaremos de *spring-mvc-serlet.xml*, ao qual iremos adicionar o suporte ao *framework*:

spring-mvc-servlet.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xmlns:mvc="http://www.springframework.org/schema/mvc"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context
        http://www.springframework.org/schema/context/spring-context.xsd
        http://www.springframework.org/schema/mvc
        http://www.springframework.org/schema/mvc/spring-mvc.xsd">

    <context:component-scan base-package="com.cruzeiro" />

</beans>
```

Figura 4 – Configuração do xml (pacotes)
Fonte: Reprodução

Ainda no mesmo arquivo, iremos acrescentar o suporte para conversão, formatação e validação:

*spring-mvc-servlet.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xmlns:mvc="http://www.springframework.org/schema/mvc"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context
        http://www.springframework.org/schema/context/spring-context.xsd
        http://www.springframework.org/schema/mvc
        http://www.springframework.org/schema/mvc/spring-mvc.xsd">

    <context:component-scan base-package="com.cruzeiro" />

    <mvc:annotation-driven/>

</beans>
```

Figura 5 – Configuração do xml (conversão)

Fonte: Reprodução

E, por fim, configurar o projeto no qual o arquivo irá procurar os arquivos. Nesse ponto, iremos configurar o *Spring* MVC *View Resolver*:

spring-mvc-servlet.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xmlns:mvc="http://www.springframework.org/schema/mvc"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context
        http://www.springframework.org/schema/context/spring-context.xsd
        http://www.springframework.org/schema/mvc
        http://www.springframework.org/schema/mvc/spring-mvc.xsd">

    <context:component-scan base-package="com.cruzeiro" />

    <mvc:annotation-driven/>

    <bean
        class="org.springframework.web.servlet.view.InternalResourceViewResolver">
        <property name="prefix" value="/WEB-INF/view/" />
        <property name="suffix" value=".jsp" />
    </bean>

</beans>
```

Figura 6 – Configuração do *xml* (*View*)

Fonte: Reprodução

Vamos fazer um passo a passo para criarmos um projeto utilizando o *Spring* MVC:

- **Passo 1:** criar um projeto *Web Dinamic Project* e dar o nome de *spring_mvc*;
- **Passo 2:** alterar a perspectiva para *Java* EE;

- **Passo 3:** copiar os arquivos .jar;
- **Passo 4:** criar um pacote **com.cruzeiro**;
- **Passo 5:** criar uma pasta no *WEB-INF* chamada *view*.

Criando *Spring Controllers* e *Views*

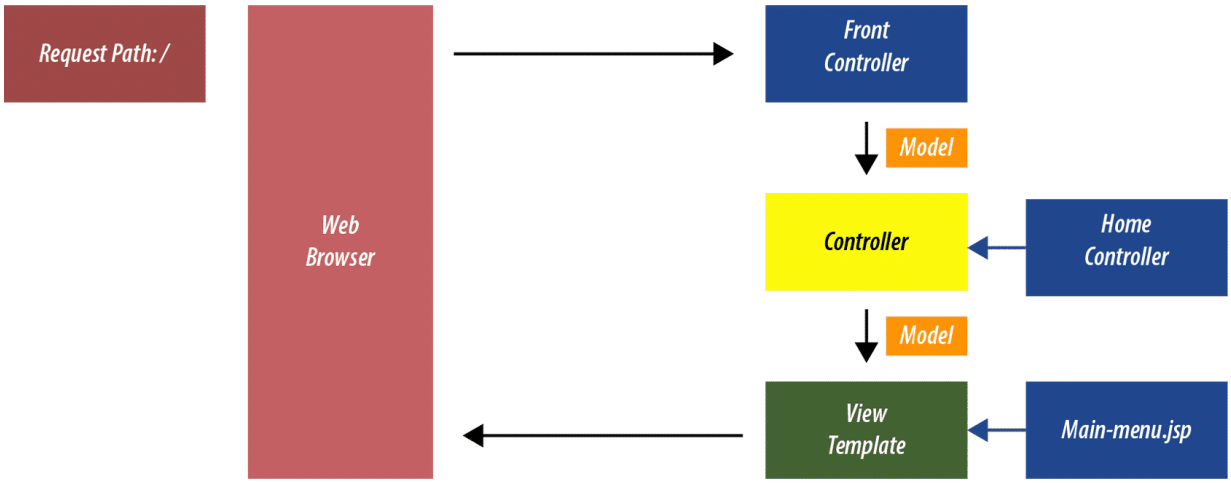


Figura 7 – Estrutura de envio dos elementos

Criar a Classe do *Controller*

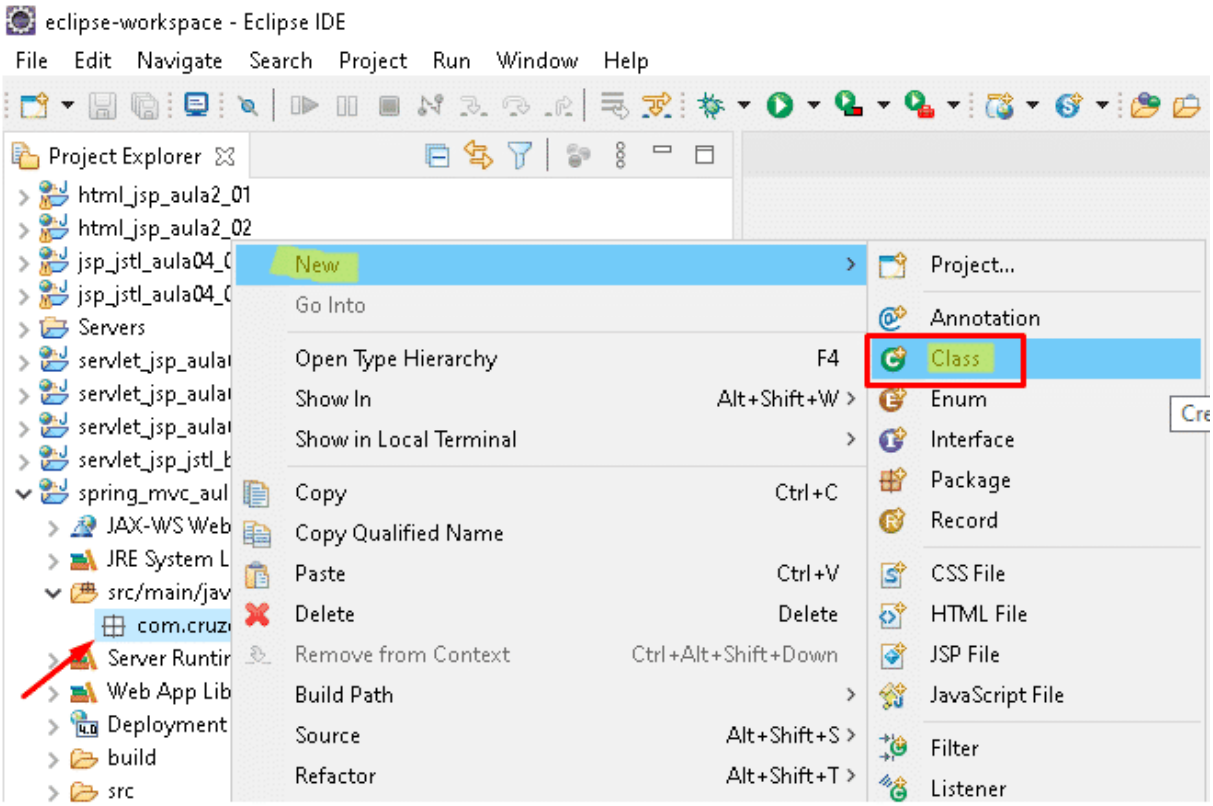


Figura 8 – Criando a classe *controller*
Fonte: Reprodução

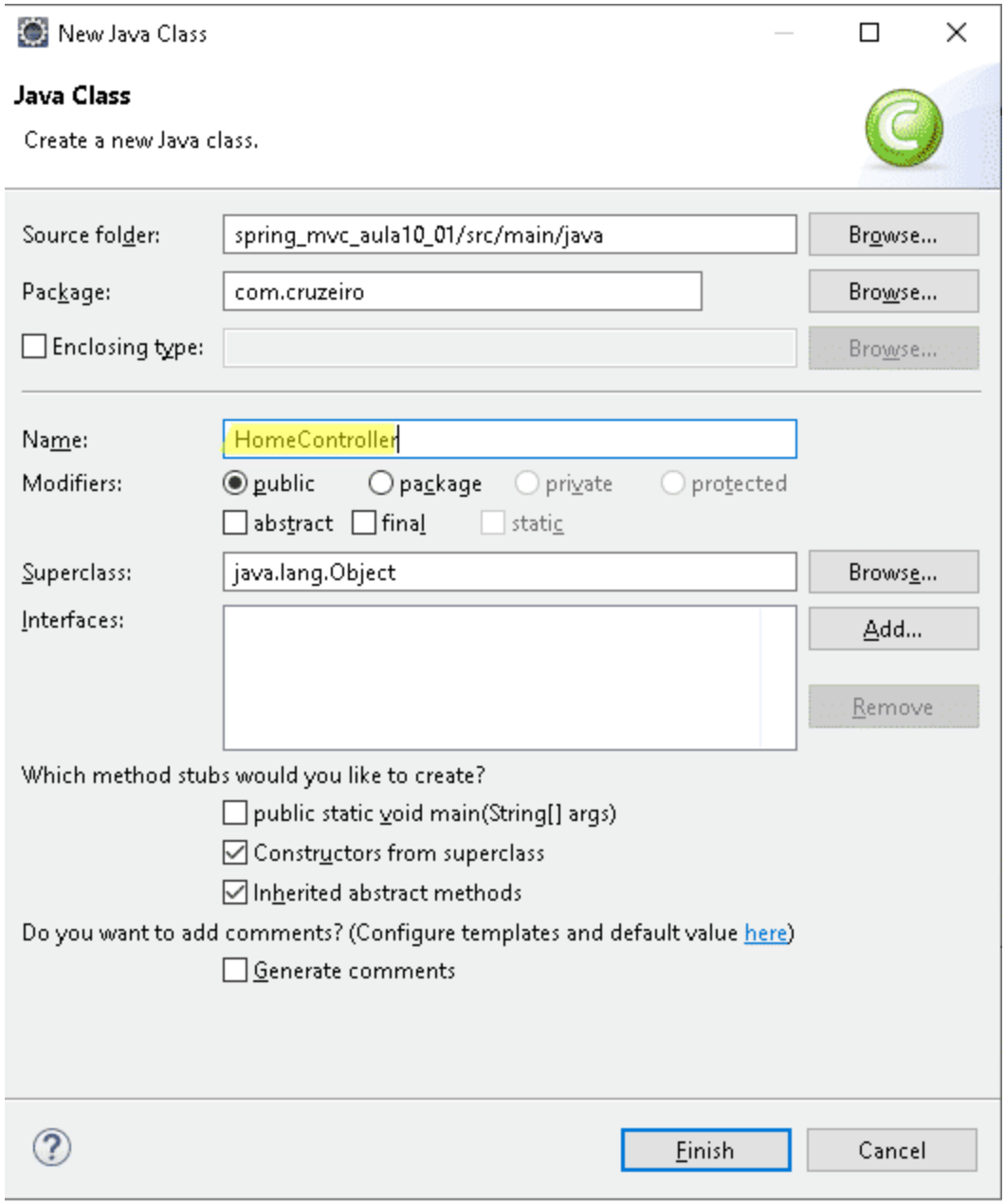


Figura 9 – Criando a *HomeController*
Fonte: Reprodução

*HomeController.java

```
1  package com.cruzeiro;
2
3  import org.springframework.stereotype.Controller;
4
5  @Controller
6  public class HomeController {
7
8      public HomeController() {
9          // TODO Auto-generated constructor stub
10     }
11
12 }
13
```

Figura 10 – Codificando a *HomeController*
Fonte: Reprodução

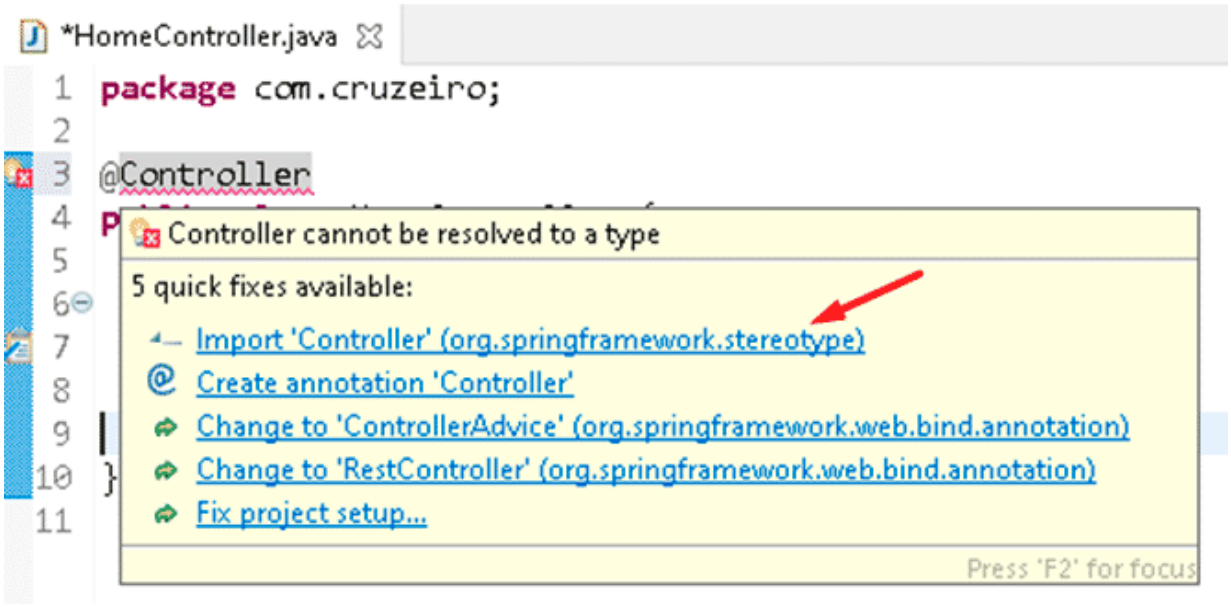


Figura 11 – Importando os pacotes
Fonte: Reprodução

Definir o Método do *Controller*

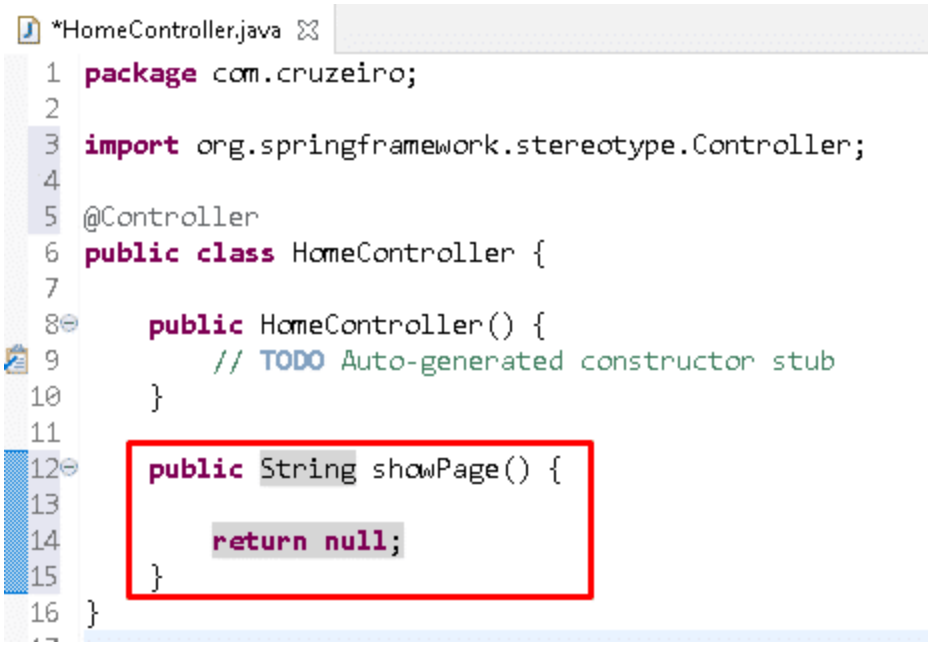


Figura 12 – Codificando a chamada
Fonte: Reprodução

Adicionar o Mapeamento da Requisição para o Método do Controle

*HomeController.java

```java
package com.cruzeiro;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
public class HomeController {

    public HomeController() {
        // TODO Auto-generated constructor stub
    }

    @RequestMapping("/")
    public String showPage() {

        return null;
    }
}
```

Figura 13 – Mapeando o método
Fonte: Reprodução

Retornar o Nome da *View*

*HomeController.java

```java
package com.cruzeiro;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
public class HomeController {

    public HomeController() {
        // TODO Auto-generated constructor stub
    }

    @RequestMapping("/")
    public String showPage() {
        return "main-menu";
    }
}
```

Figura 14 – Definindo o nome da página de mapeamento
Fonte: Reprodução

Desenvolver a Página de Visualização

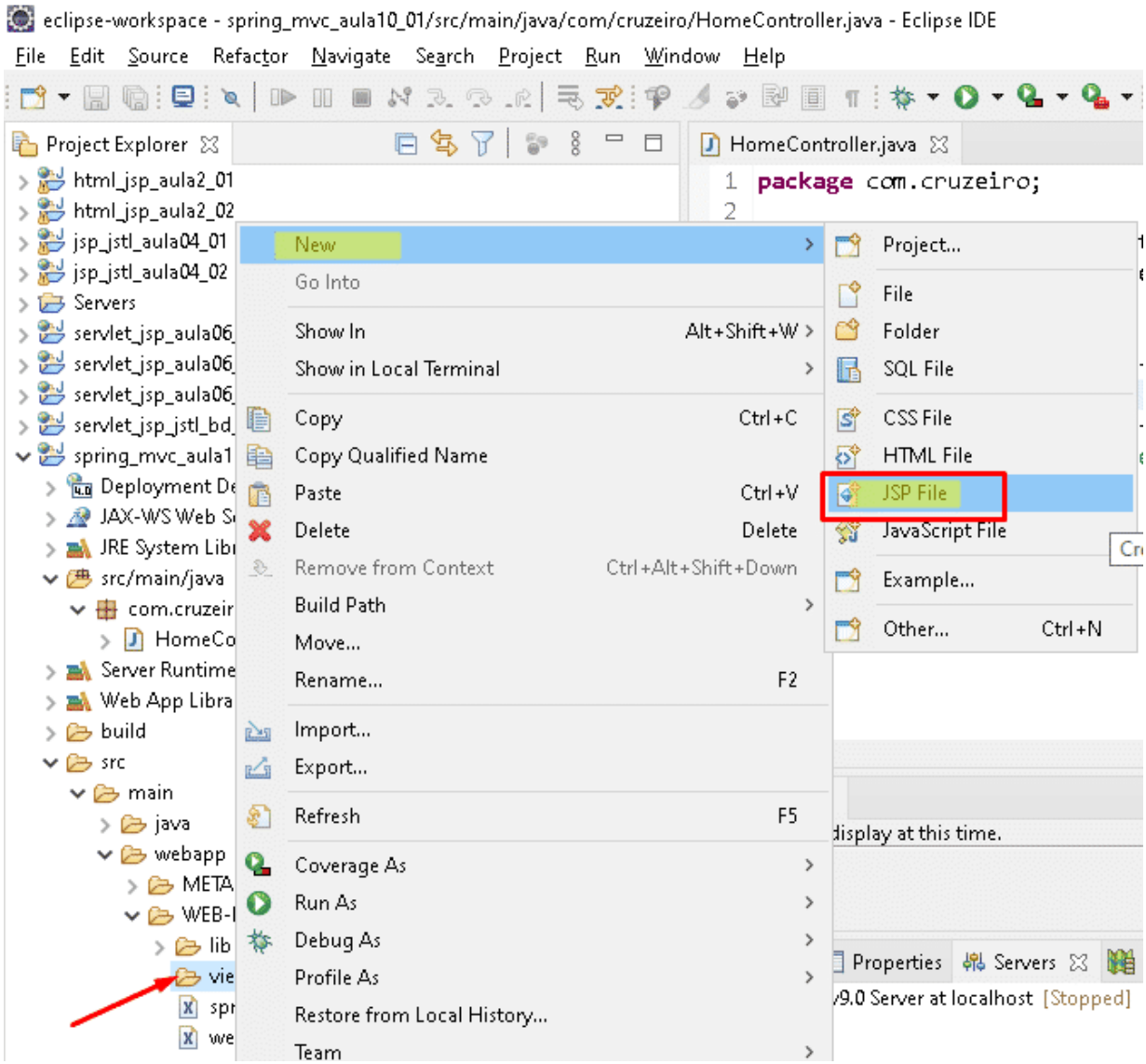


Figura 15 – Criando a página de visualização
Fonte: Reprodução

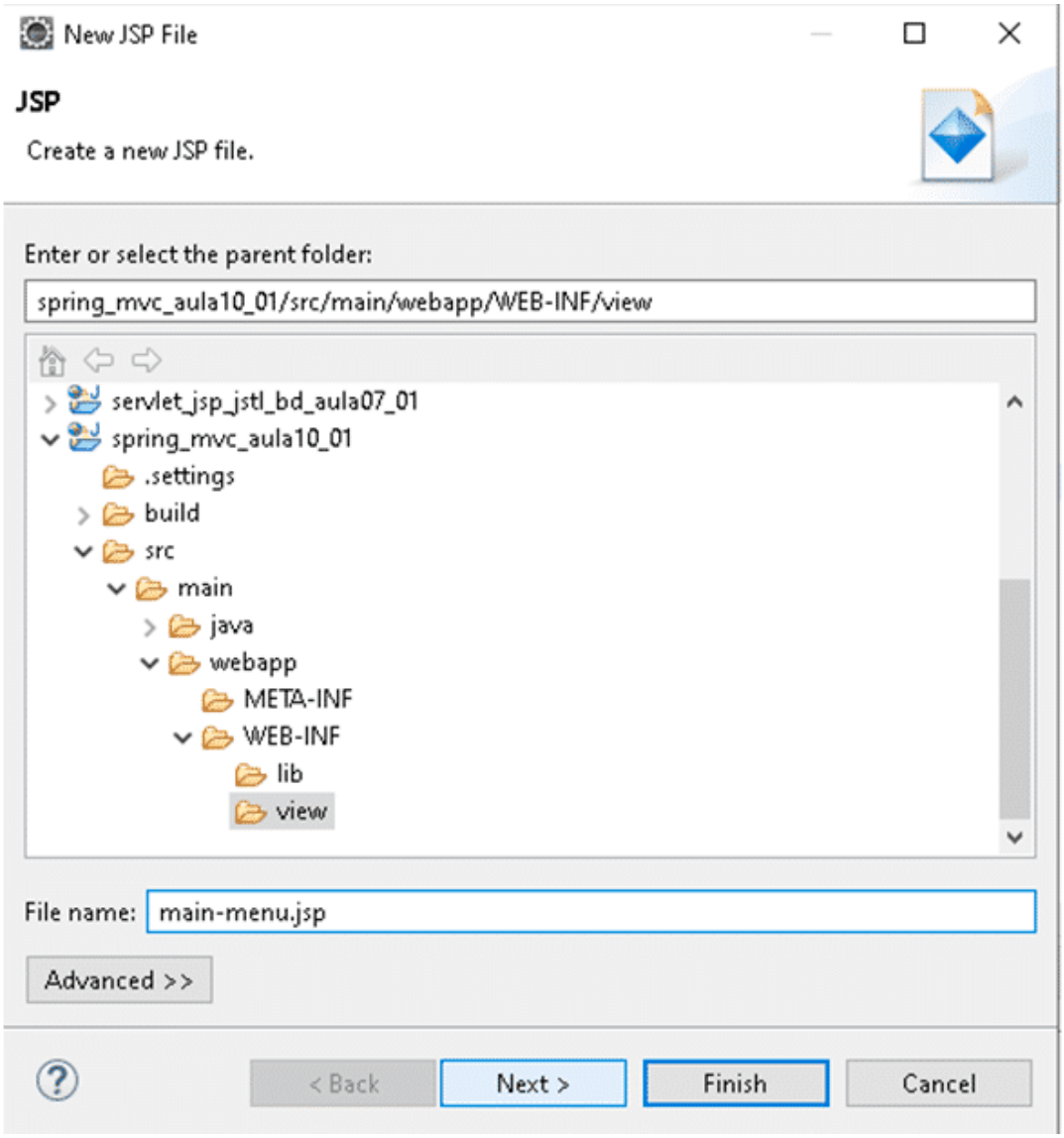


Figura 16 – Definindo o nome da página
Fonte: Reprodução

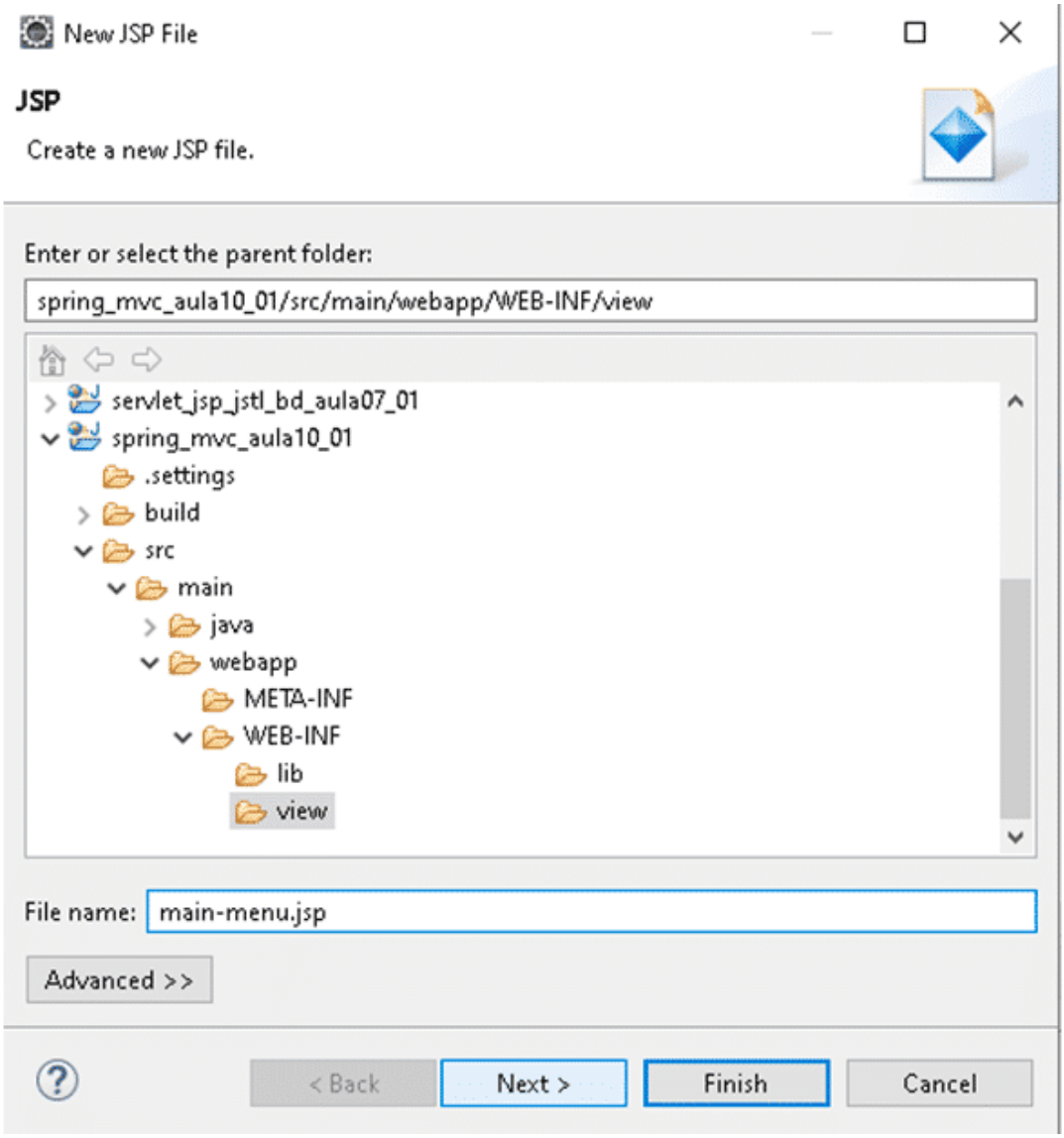


Figura 17 – Selecionando a pasta
Fonte: Reprodução



Figura 18 – Resultado da execução
Fonte: Reprodução

Spring MVC *Forms Tags*

O *Spring* tem suporte para *tags* de formulário e elas são *tags* de formulário configuráveis, sendo que você pode reutilizá-las em uma página da *Web*.

Essas *tags* de formulário do *Spring* MVC oferecem um suporte extra, que suportam ligação de dados. Portanto, isso permite que você configure dados automaticamente e recupere dados de

objetos e *beans Java*.

Essas *tags* de formulário realmente gerarão html para você. Assim, farão muito trabalho pesado nos bastidores.

Lista das *tags*:

- *Tag* de formulário principal;
- *Tag* de entrada para campos de texto;
- *Tag* área de texto para campos de texto com várias linhas;
- *Tag* caixa de seleção;
- *Tag* botões de opção;
- *Tag* de seleção para fazer uma lista suspensa.

Basicamente, ele faz uso de uma página jsp normal e depois simplesmente solta as *tags* de formulário *Spring MVC*.

Então, você tem suas coisas html regulares, *tags* de formulário do *Spring* MVC e mais html e pode facilmente misturá-los ao seu conteúdo existente na página da *web*.

Fazendo Referência a essas *Tags* no Arquivo JSP

No começo do seu arquivo jsp, você deve fazer uso da *tag lib reference* e, uma vez que você tenha configurado, você poderá usar as *tags* na página JSP.

Spring MVC *Form Tag – Text Field*

<*form:input*> – Representa uma *tag* de entrada HTML usada para desenvolver campos de texto utilizando valores vinculados.

Spring MVC Form Tag – DropDown List

<*form:select*> – Representa a *tag* HTML <*select*>, que apresenta uma lista de itens.

Ele também suporta a *tag* aninhada **<*form:option*>** e **<*form:options*>**.

Spring MVC Form Tag – RadioButton

A *tag* **<*form:radiobutton*>** é usada para criar um campo de botão de rádio HTML no qual os valores são codificados dentro das páginas JSP, enquanto a *tag* **<*form:radiobuttons*>** é usada para criar vários botões de rádio nos quais os valores são gerados em o tempo de execução.

Spring MVC Form Tag – Checkbox

A *tag* **<*form:checkbox*>** é usada para criar um campo de caixa de seleção HTML no qual os valores são codificados dentro das páginas JSP, enquanto a *tag* **<*form:checkboxes*>** é usada para criar várias caixas de seleção nas quais os valores das caixas de seleção são gerados em tempo de execução.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Livros

Aprenda J2EE em 21 Dias

BOND, M. *et al*. **Aprenda J2EE em 21 dias**. Pearson 1000, 2003. (*e-book*)

Internet Applications e Desenvolvimento *Web* para Programadores

DEITEL, P. J.; DEITEL, H. M. AJAX, R. ***Internet applications* e desenvolvimento *web* para programadores**. Pearson 776, 2008. (*e-book*)

Lógica de Programação e Estrutura de Dados: com aplicações em *Java*

PUGA, S.; RISSETTI, G. **Lógica de programação e estrutura de dados:** com aplicações em *Java*. 2. ed. Pearson 266, 2017. (*e-book*)

Core Java

HORSTMANN, C. S.; CORNELL, G. ***Core Java***. 8. ed. Pearson 402. v. 1. 2007. (*e-book*)

Referências

OVERVIEW of Java Server Pages (*JSP*). ***Oracle***, 2006. Disponível em: <https://docs.oracle.com/cd/E13222_01/wls/docs81/jsp/intro.html>. Acesso em 07/07/2022.

SPRING MVC form example. ***Java T Point***, [s. d.]. Disponível em: <https://www.javatpoint.com/spring-mvc-form-example>. Acesso em 07/07/2022.